

Projeto To Do AWS

Relatorio Tecnico

Aluno: Eduardo Pirolo

Data: 26/05/2026

Repositorio: github.com/eduardocardozopirolo/to_do_list

Aplicacao CRUD de tarefas com backend Node.js/Express, Docker, Amazon EC2, Amazon RDS PostgreSQL, AWS Lambda, API Gateway e frontend web em HTML, CSS e JavaScript.

1. Objetivo

O projeto demonstra uma aplicacao CRUD completa em ambiente AWS. O usuario pode criar, listar, atualizar e excluir tarefas. Os dados sao persistidos em um banco PostgreSQL gerenciado pelo Amazon RDS.

Alem do CRUD, uma funcao AWS Lambda gera um relatorio com total de tarefas, tarefas concluidas e tarefas pendentes. Essa funcao e exposta por uma rota GET /report no API Gateway.

2. Arquitetura

Fluxo principal:

Usuario -> Frontend HTML/CSS/JS -> Backend Node.js/Express na EC2 -> Amazon RDS PostgreSQL

Fluxo do relatorio:

Usuario -> API Gateway GET /report -> AWS Lambda -> Backend GET /tasks -> Amazon RDS PostgreSQL

Essa separacao mostra o uso de EC2 para hospedagem de containers, RDS para persistencia, Lambda para processamento sob demanda e API Gateway para exposicao de endpoint serverless.

3. Componentes Utilizados

Frontend: interface web estatica para consumir a API, executar CRUD e mostrar relatorio.

Node.js + Express: backend REST responsavel pelas rotas de tarefas.

Docker: empacotamento do backend e frontend em containers.

EC2: hospedagem dos containers da aplicacao.

RDS PostgreSQL: banco relacional gerenciado para persistencia das tarefas.

Lambda: funcao responsavel por consultar /tasks e calcular estatisticas.

API Gateway: exposicao publica da rota GET /report integrada a Lambda.

4. Modelo de Dados

Tabela principal: tasks.

Campos: id, title, description, status, created_at.

SQL utilizado:

```
CREATE TABLE tasks (  
  id SERIAL PRIMARY KEY,  
  title VARCHAR(255),  
  description TEXT,  
  status VARCHAR(50),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

O campo status identifica tarefas pendentes ou concluidas. A Lambda considera completed, complete, done, concluido e concluida como status de conclusao.

5. Backend e Rotas

O backend foi implementado com Express. A conexao com o PostgreSQL fica centralizada em src/db.js, usando variaveis de ambiente para manter credenciais fora do codigo.

GET /tasks: lista todas as tarefas.

POST /tasks: cria uma tarefa.

PUT /tasks/{id}: atualiza uma tarefa existente.

DELETE /tasks/{id}: remove uma tarefa.

GET /report: relatório exposto pelo API Gateway e Lambda.

Exemplo de POST /tasks: { title: 'Teste RDS', description: 'Criado pela API', status: 'pending' }.

6. RDS PostgreSQL

O Amazon RDS hospeda o PostgreSQL usado pela aplicação. O backend acessa o banco pela porta 5432 com SSL habilitado.

Variáveis principais:

DB_HOST=tododb.cozyt5thfmg.us-east-1.rds.amazonaws.com

DB_USER=postgres

DB_NAME=postgres

DB_PORT=5432

DB_SSL=true

A senha real fica somente no arquivo .env da EC2 e não deve ser publicada no GitHub.

Comando de validação: `psql -h tododb.cozyt5thfmg.us-east-1.rds.amazonaws.com -U postgres -d postgres -p 5432`

Query de validação: `SELECT * FROM tasks ORDER BY id DESC;`

7. Docker e Deploy na EC2

O backend e o frontend foram preparados para execução em Docker. No backend, o container executa a API Node.js na porta 3000. No frontend, o Nginx serve os arquivos estáticos na porta 80 do container.

Backend: `cd project/backend; docker compose up -d --build; docker logs todo-api.`

Frontend: `cd project/frontend; docker build -t todo-frontend .; docker run -d --name todo-frontend --restart always -p 8080:80 todo-frontend.`

O parâmetro `--restart always` garante que o container do frontend reinicie automaticamente caso a EC2 seja reiniciada.

8. Lambda e API Gateway

A Lambda fica em `project/lambda/report/index.mjs`. Ela busca as tarefas no endpoint configurado em `TASKS_API_URL` e retorna um JSON com estatísticas.

Nome: `lamdaapitodo`

Runtime: `Node.js 20.x`

Handler: `index.handler`

`TASKS_API_URL=http://IP_DA_EC2_BACKEND:3000/tasks`

Resposta esperada: { total: 5, completed: 2, pending: 3, generated_at: '...' }.

No API Gateway, a rota GET /report deve estar integrada diretamente com a Lambda.

9. Frontend

O frontend foi criado em HTML, CSS e JavaScript puro. Ele permite configurar a URL da API de tarefas e a URL do relatório.

Funcionalidades: listar tarefas, criar tarefas, editar tarefas, concluir tarefas, excluir tarefas e visualizar cards de total, concluídas e pendentes.

URL da API de tarefas: http://IP_DA_EC2_BACKEND:3000

URL do relatorio: <https://SUA-API-GATEWAY.execute-api.us-east-1.amazonaws.com/report>

10. Seguranca e Redes

EC2 Backend: SSH 22 do IP do aluno e TCP 3000 publico para testes.

RDS: PostgreSQL 5432 somente do Security Group da EC2 backend.

EC2 Frontend: SSH 22 do IP do aluno e TCP 8080 ou HTTP 80 publico.

Lambda: se estiver fora da VPC e chamando a EC2 publica, nao precisa de Security Group proprio.

Em producao, o backend nao deveria ficar exposto diretamente na porta 3000. O ideal seria usar Load Balancer ou API Gateway com VPC Link.

11. Testes e Validacao

1. GET /tasks retorna uma lista JSON.
2. POST /tasks cria uma tarefa e retorna um id.
3. SELECT * FROM tasks no RDS confirma persistencia.
4. PUT /tasks/{id} altera uma tarefa.
5. DELETE /tasks/{id} remove uma tarefa.
6. GET /report retorna total, completed, pending e generated_at.
7. Frontend consome backend e relatorio corretamente.

12. Conclusao

O projeto entrega uma aplicacao distribuida funcional com persistencia em banco gerenciado, backend containerizado, frontend separado, Lambda para relatorio e API Gateway para endpoint serverless.

Como melhoria futura, o backend pode ser colocado atras de um Application Load Balancer ou API Gateway com VPC Link, removendo o acesso publico direto a porta 3000 da EC2.